NEBIUS ACADEMY

RAG, Workflows & AI Systems

It's not an AI agent unless it comes from the agentic region of Palo Alto. Otherwise, it's just an AI-enabled workflow

Yuval Belfer

April 2026

© Steven Kazlowski | www.leffeyepro.com
imgflip.com

Today's Agenda

01
Why RAG?
The finetuning misconception & when to use RAG

02
RAG from Scratch
Pre-processing · Chunking · Embeddings · Retrieval · Generation

03
RAG Alternatives & OOTB
Long context, fine-tuning as alternative, frameworks

04
Evaluating RAG Systems
Metrics, LLM-as-judge, live notebook demo

05
Workflows & AI Systems
Tool use, chaining, routing, parallelization

06
Agents — Teaser
What they are, when NOT to use them · deep dive next lecture

Why RAG?

The finetuning misconception

Problems that RAG solves

01

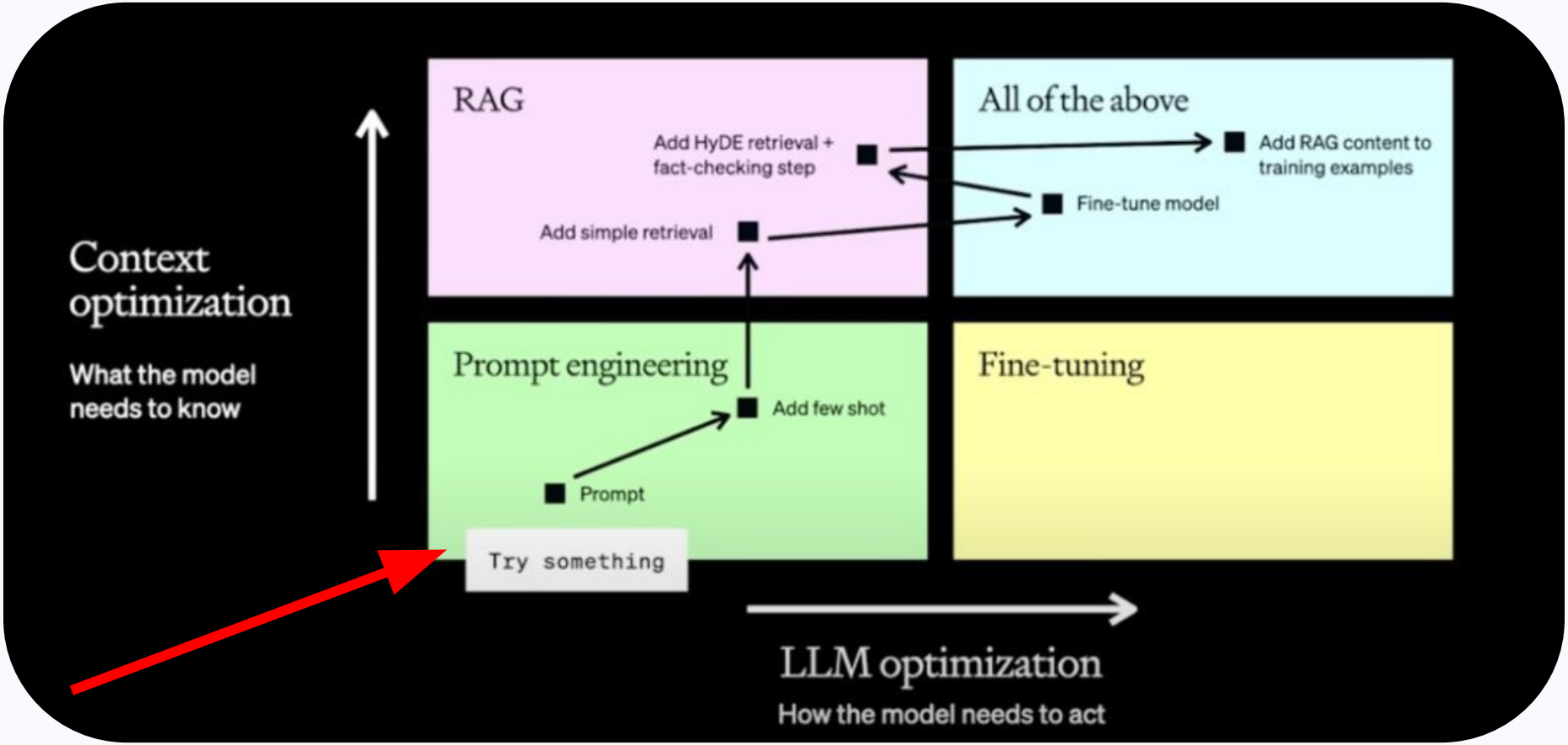
The Data Problem

- Not all data is public
- Not all public data is reliable
- There is just too much data

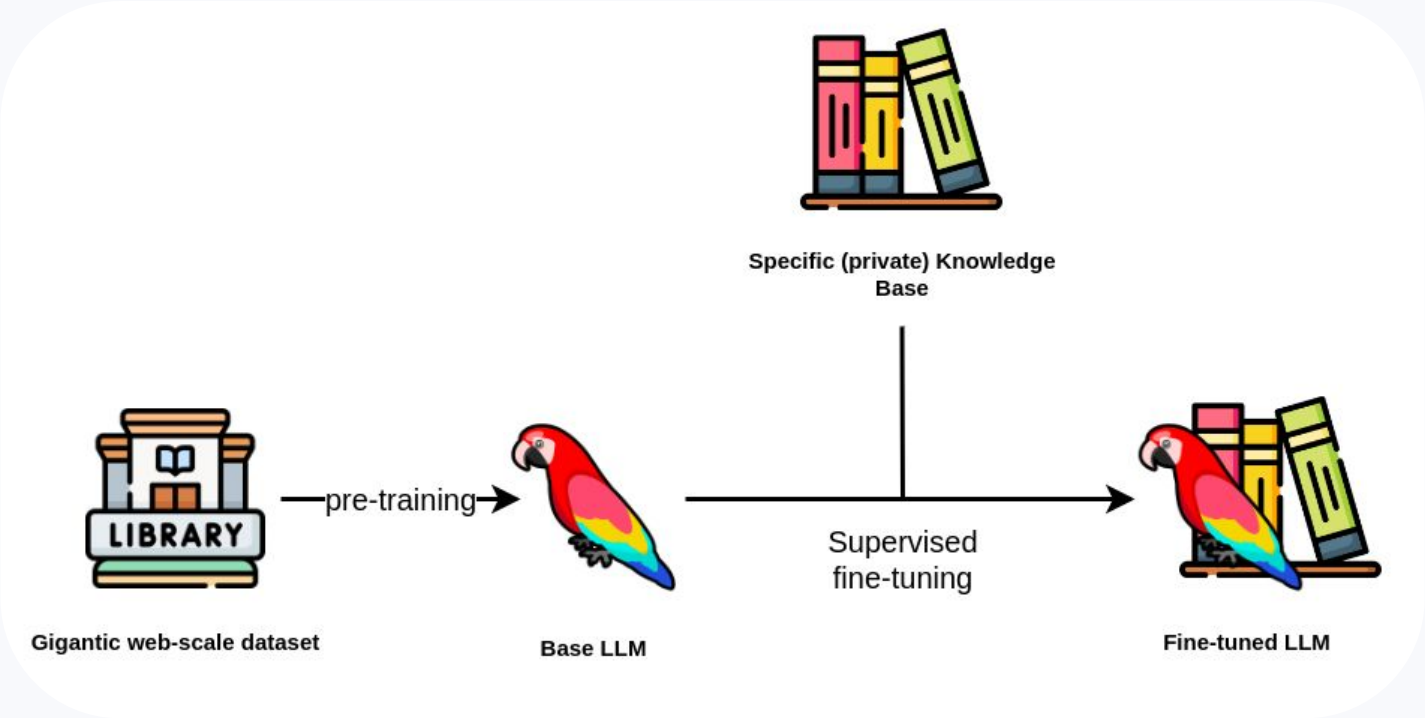
02

The Model Problem

- Knowledge cut-off date
- Always 'tempted' to answer
- Context ↑ → performance ↓
- Context window size limit

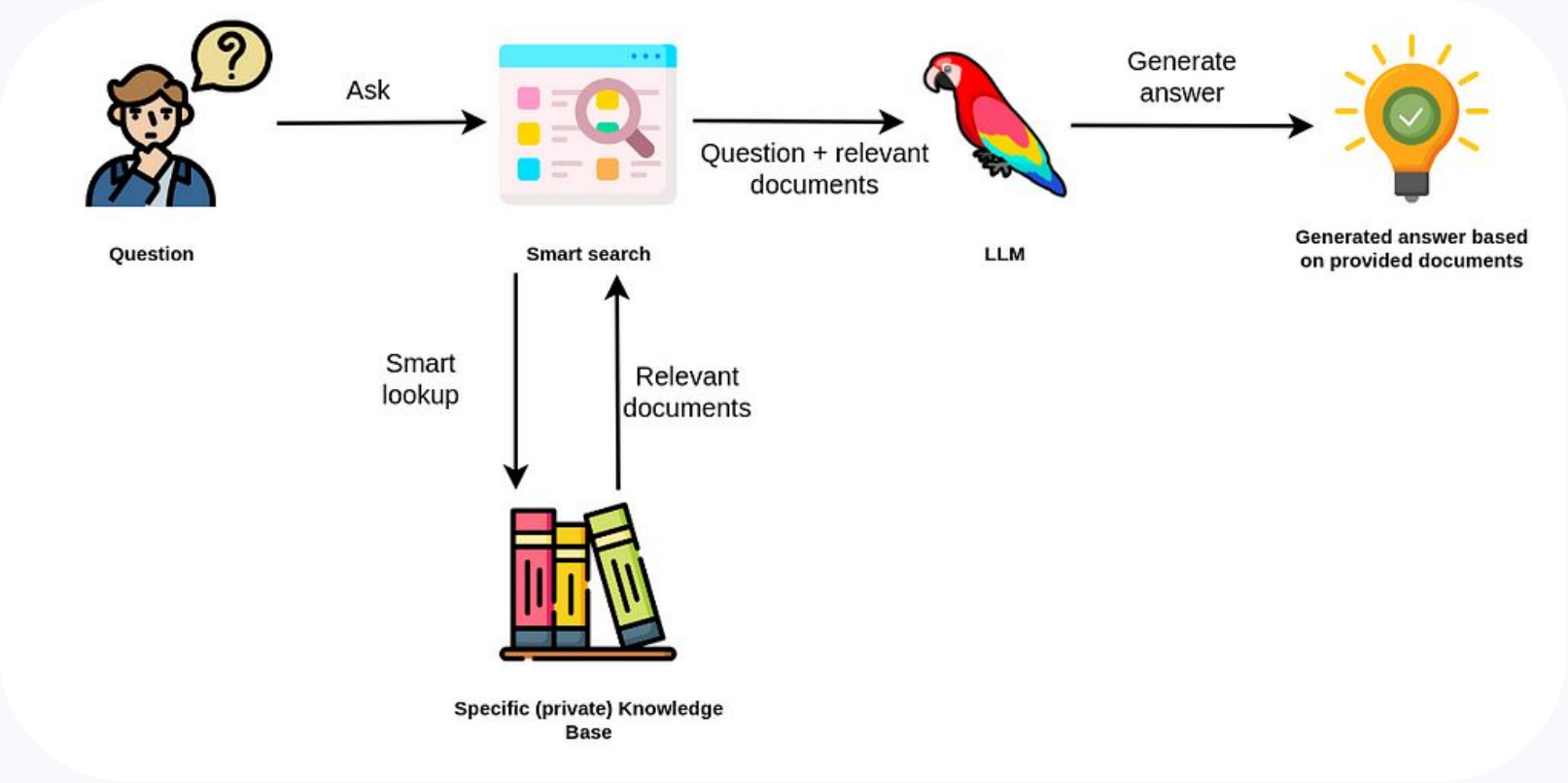

RAG
Add HyDE retrieval + fact-checking step
Add simple retrieval
All of the above
Add RAG content to training examples
Fine-tune model
Context optimization
What the model needs to know
Prompt engineering
Add few shot
Prompt
Try something
Fine-tuning
LLM optimization
How the model needs to act

Fine-tuning



https://neo4j.com/developer-blog/fine-tuning-retrieval-augmented-generation/

RAG



https://neo4j.com/developer-blog/fine-tuning-retrieval-augmented-generation/

“I have a lot of data, so I want to finetune a model”

Every customer, every call, 2024-2025

“I have a lot of data, so I want to finetune a model”

Every customer, every call, 2024-2025

Having lots of data does not mean you need finetuning.

"I have a lot of data, so I want to finetune a model"

Every customer, every call, 2024-2025

The right question:

Do you need the model to write differently, or to know more?

Finetune vs. RAG - the one-liner rule

Finetune

"The model should write like..."

Style, tone, format, domain vocabulary.
You want to change HOW the model writes.

🦜 **Finetune:** Parrot who spent a lot of time in the library reading books in a specific subject

RAG

"The model should write about..."

Private data, recent facts, dynamic knowledge.
You want to change WHAT the model knows.

🦜 **RAG:** Parrot in the library with a good search mechanism

RAG from Scratch

Building it step by step

What is RAG?

RAG = Retrieval Augmented Generation

A technique that enhances the accuracy and reliability of LLM-generated responses by combining the model with an external, up-to-date knowledge source.

Without RAG

→ Model relies on training data only

→ Knowledge frozen at cutoff date

→ Hallucinations when data is missing

→ Cannot access private data

With RAG

→ Retrieves relevant docs at query time

→ Always up-to-date knowledge

→ Grounded answers — lower hallucination

→ Works with any private knowledge base

The RAG Basic Recipe

1

CHUNK

Segment your data into digestible pieces

2

EMBED

Transform chunks into N-dimensional numeric vectors

3

RETRIEVE

Use the query to find the most similar chunks

4

GENERATE

Pass query + retrieved chunks to LLM → grounded answer

The RAG Basic Recipe

INDEXING (Offline)

1 CHUNK

Segment your data into digestible pieces

2 EMBED

Transform chunks into N-dimensional numeric vectors

3 RETRIEVE

Use the query to find the most similar chunks

4 GENERATE

Pass query + retrieved chunks to LLM → grounded answer

The RAG Basic Recipe

INDEXING (Offline)

INFERENCE (Query time)

1 CHUNK

Segment your data into digestible pieces

2 EMBED

Transform chunks into N-dimensional numeric vectors

3 RETRIEVE

Use the query to find the most similar chunks

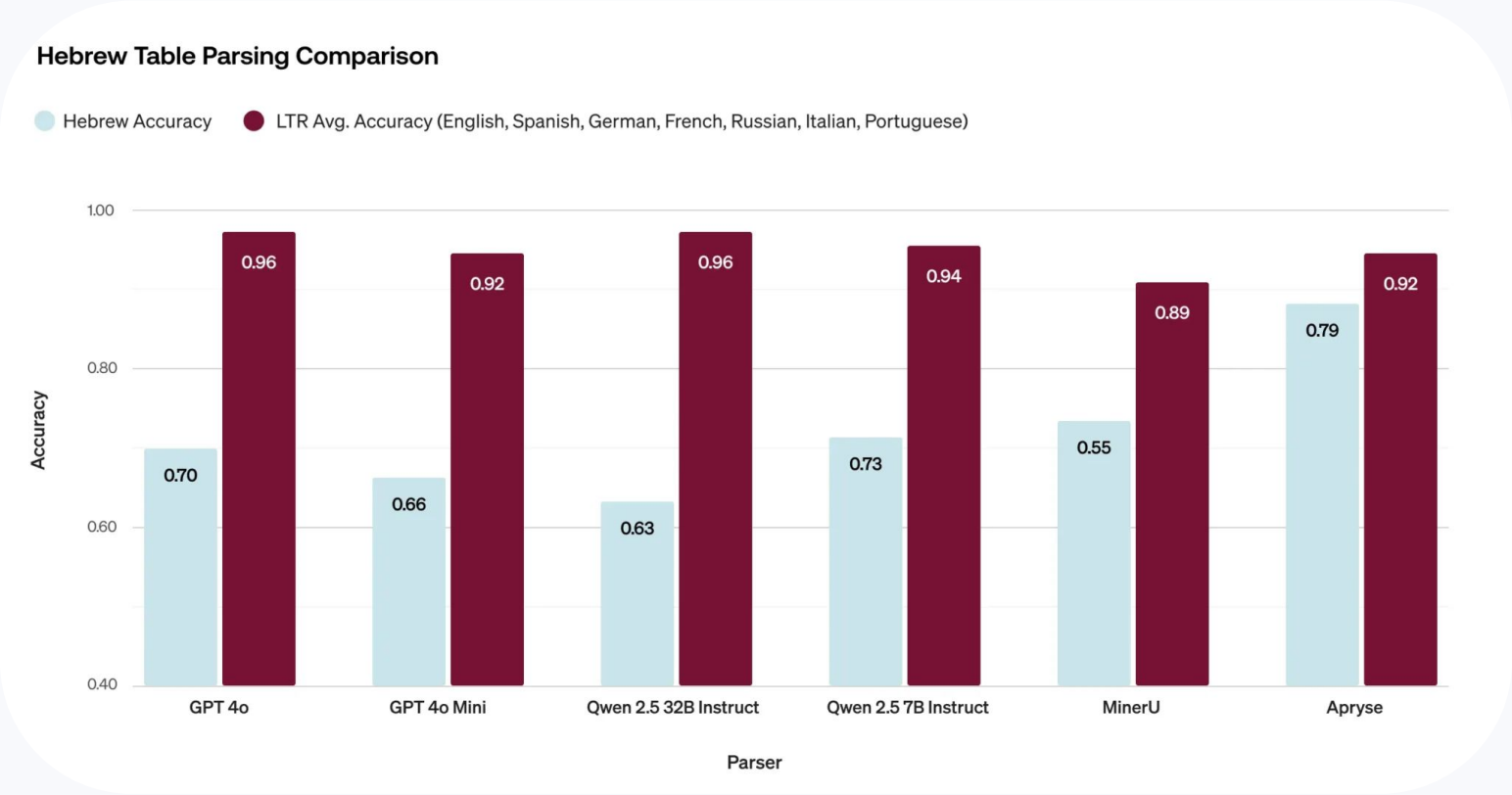
4 GENERATE

Pass query + retrieved chunks to LLM → grounded answer

Step 0: Pre-processing

- Converting any raw content (PDFs, Word docs, HTML, tables, images) into clean text that an LLM can read, while preserving structure and meaning.
- **It's 2026 and parsing a PDF correctly is still not trivial**
- Popular parsers: llamaparse, Amazon Textract, …
- What about L2R parsing?

Tangent: L2R Parsing



Step 1: Chunking

We split documents into pieces so semantic search can retrieve them precisely.

Step 1: Chunking

01 **Semantic clarity**

Smaller, coherent chunks produce better embeddings that capture specific topics. One embedding for a huge document dilutes meaning across many subtopics.

02 **Ranked granular results**

Chunk-level embeddings let you rank individual passages and retrieve exactly the matching segment — not an entire document.

03 **Flexible retrieval strategies**

Enables hybrid search, re-ranking, neighbour expansion, and multi-hop reasoning. All of these require chunk-level granularity.

Step 1: Chunking

We split documents into pieces so semantic search can retrieve them precisely.

Large chunk size

Fixed chunk size

Adaptive chunk size

Step 1: Chunking

What is better?

We split documents i c search can retrieve them precisely.

Large chunk size

Fixed chunk size

Adaptive chunk size

Step 1: Chunking (Level Up)

01 **Logical chunking**

Chunk at natural boundaries: paragraphs, sections, sentences — not fixed character counts. Context should stay intact within each chunk.

02 **Overlapping chunks**

Add 10–20% overlap between adjacent chunks. Prevents critical information from being severed at a boundary.

03 **Data enhancement / text planting**

Plant hints into chunks before embedding: section hierarchy, metadata, query-like prefixes. Anthropic calls this 'contextual retrieval'. Ori Shapira invented it a year earlier and calls it text planting.

Step 1: Chunking (Level Up)

01 **Logical chunkin**

Ch

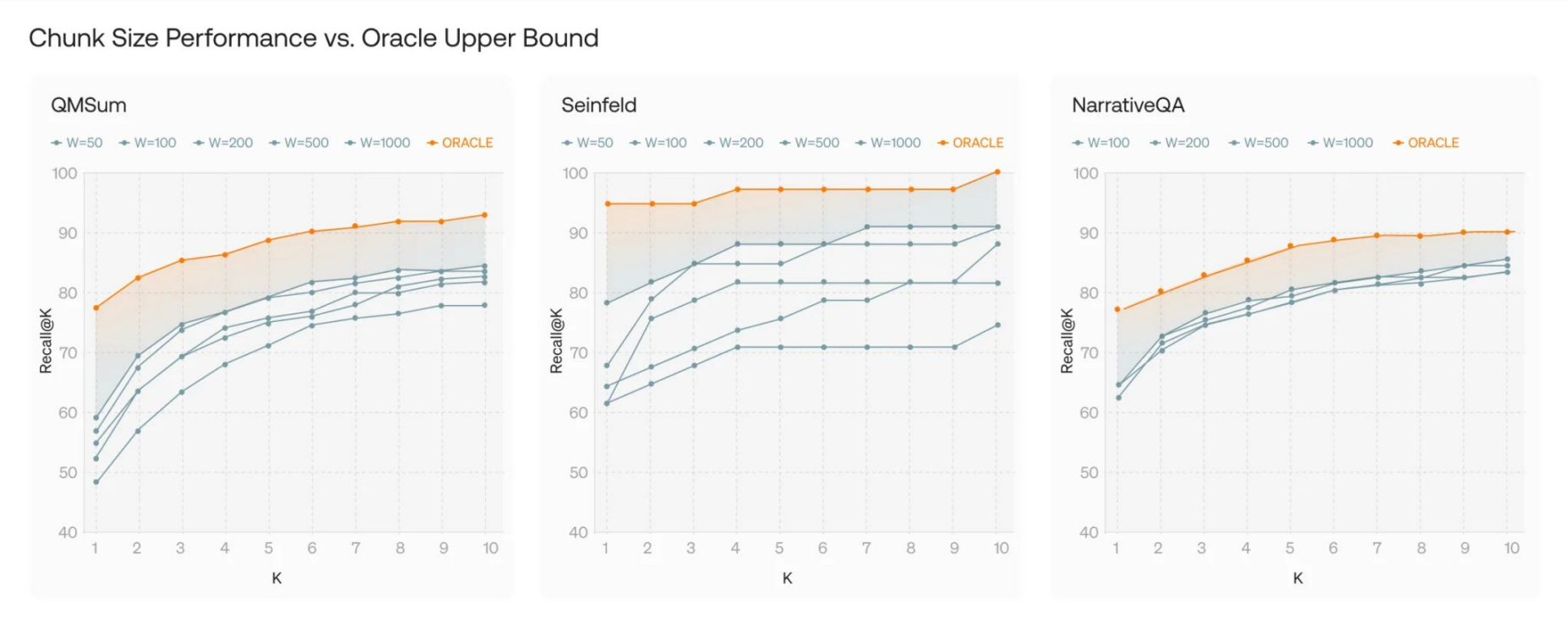
0

03 **Data**

Plant hints into c
Anthropic calls this 'contextu
planting.

Common chunking strategy these days: fixed size with overlap

Step 1: Chunking (Even More Level Up)



Step 2: Embeddings & Vector DB

An embedding model is...

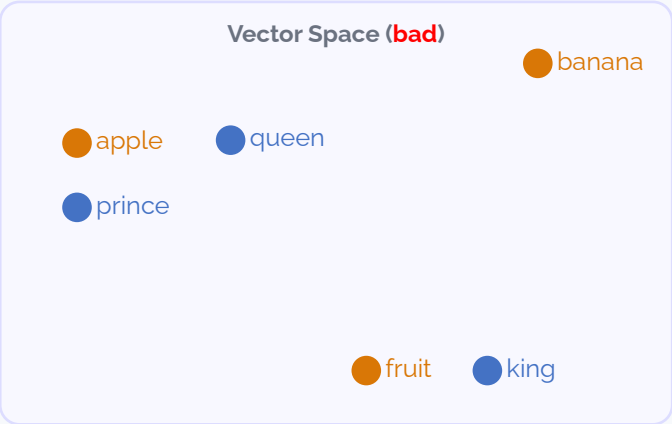
A learnable function that maps complex data (text, images) into continuous, dense vectors in a lower-dimensional space — capturing semantic and contextual meaning numerically.

Step 2: Embeddings & Vector DB

An embedding model is...

A learnable function that maps complex data (text, images) into continuous, dense vectors in a lower-dimensional space — capturing semantic and contextual meaning numerically.

Semantic proximity: similar inputs cluster together in vector space

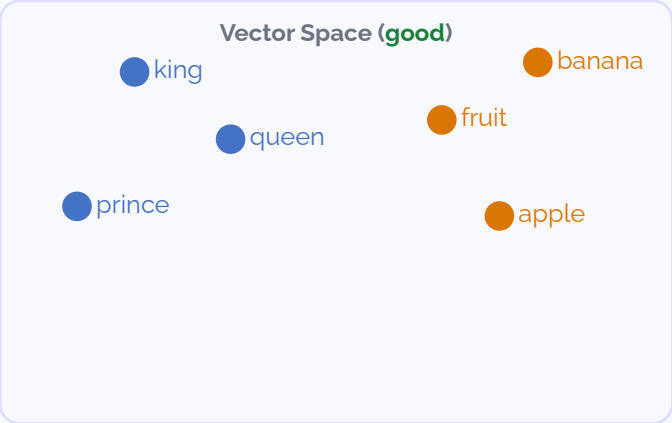


Step 2: Embeddings & Vector DB

An embedding model is...

A learnable function that maps complex data (text, images) into continuous, dense vectors in a lower-dimensional space — capturing semantic and contextual meaning numerically.

Semantic proximity: similar inputs cluster together in vector space



Similarity metrics

Cosine similarity ✓ Default
Meaning only (ignore magnitude)

Dot product
Magnitude matters

Euclidean (L2)
Geometric distance / clustering

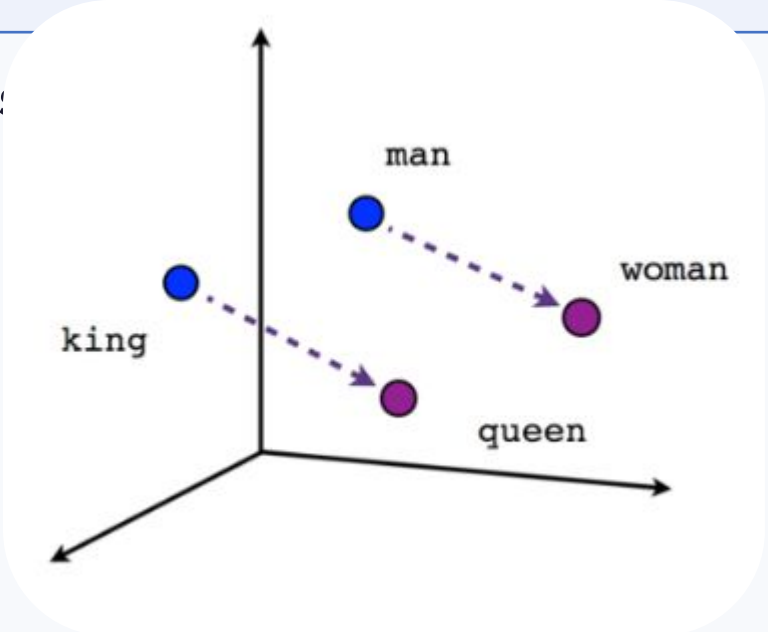
Hybrid (keyword + semantic)
When exact terms matter too

Step 2: Embeddings & Vector DB

An embedding model is...

A learnable function that maps complex data (text, images) into continuous, dense vectors in a lower-dimensional space — capturing semantic and contextual meaning numerically.

Semantic proximity: similar inputs clu



Step 2: Embeddings & Vector DB

- Stores embedding vectors alongside the original text chunks
- Supports fast Approximate Nearest Neighbor (ANN) search - you don't compare against every chunk, just "close enough"
- Popular options: ChromaDB (easy, in-memory, great for demos), Pinecone (managed, scalable), Weaviate

Step 3: Retrieval

- Convert query → embedding
- Compare against all stored vectors
- Return top-k most similar chunks

Garbage retrieval → garbage generation

Step 3: Retrieval Glossary

- Top-k (how many chunks?)
- Similarity metric (cosine, dot, L2)
- Chunk size impact
- Metadata filtering (date, source, etc.)

Step 3: Retrieval - Failure Modes

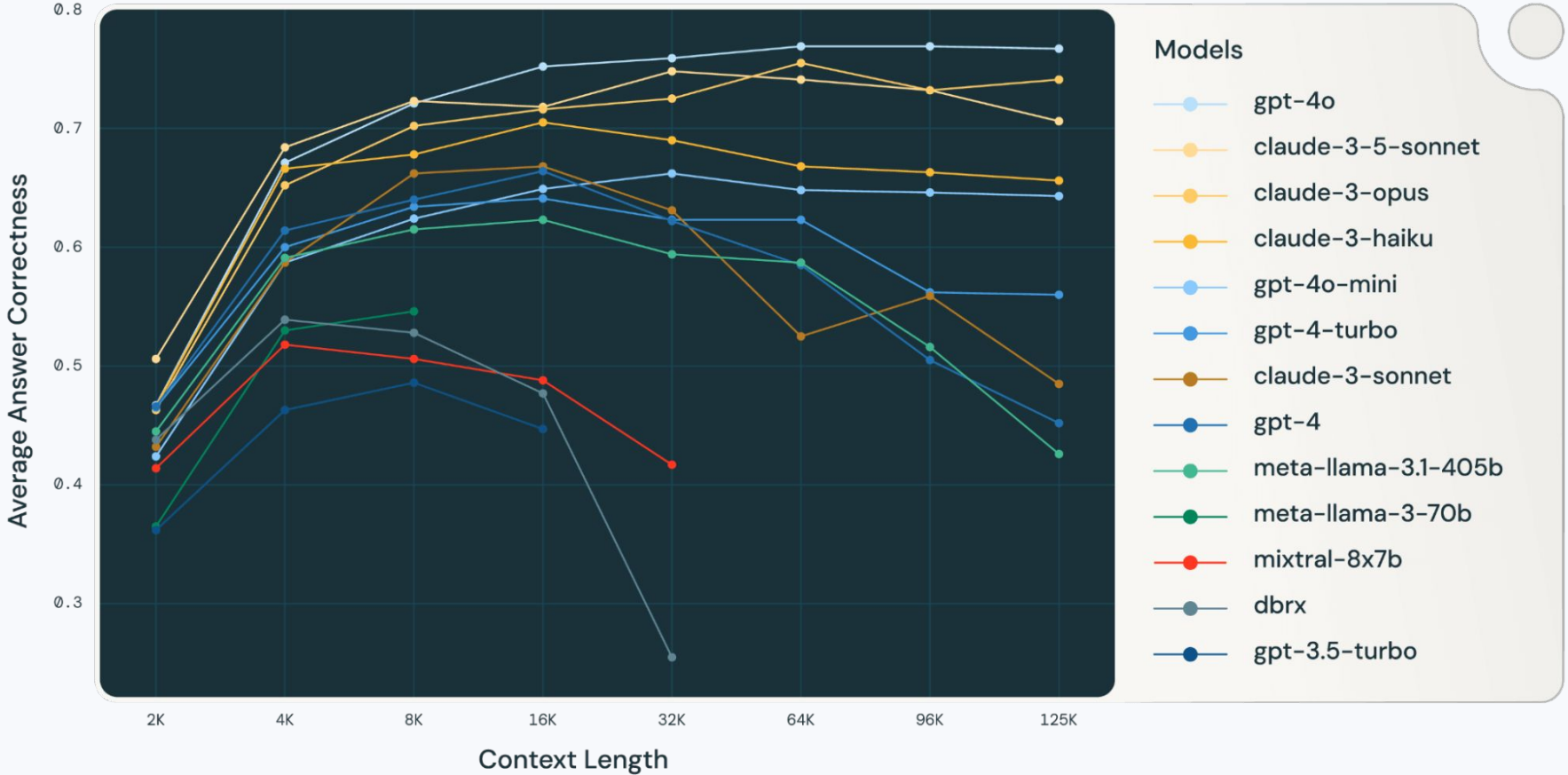
- Relevant doc not retrieved
- Retrieved docs are noisy / irrelevant
- Missing context (chunk too small)
- Wrong ranking order

Step 3: Retrieval - How To Measure?

# Retrieved chunks	1	5	13	29	61	125	189	253	317	381
Recall@k \ Context Length	2k	4k	8k	16k	32k	64k	96k	128k	160k	192k
Databricks DocsQA	0.547	0.856	0.906	0.957	0.978	0.986	**0.993**	**0.993**	**0.993**	**0.993**
FinanceBench	0.097	0.287	0.493	0.603	0.764	0.856	**0.916**	**0.916**	**0.916**	**0.916**
NQ	0.845	0.992	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**
HotPotQA	0.382	0.672	0.751	0.797	0.833	0.864	0.880	**0.890**	**0.890**	**0.890**
Average	0.468	0.702	0.788	0.839	0.894	0.927	0.947	**0.95**	**0.95**	**0.95**

https://www.databricks.com/blog/long-context-rag-performance-llms

Step 3: Retrieval - Only Recall Matters?



Step 3: Retrieval++

- Hybrid search (keyword + semantic)
- Re-ranking (cross-encoder / LLM)
- Query rewriting (“better search query”)
- Multi-hop retrieval (retrieve → expand → retrieve again)
- Neighbor expansion (retrieve adjacent chunks)
- Multi-query retrieval (ask multiple variants)

Step 4: Generation

- Input = query + retrieved chunks
- Prompt template injects context
- LLM generates answer grounded in docs

Step 4: Generation - Prompting Matters (a lot)

- “Answer ONLY based on the context”
- Cite sources
- Handle missing info (“say I don’t know”)

Retrieval gives you facts; prompting decides how they’re used

Step 4: Generation - Common Failures

- Ignores retrieved context
- Hallucinates anyway
- Over-relies on one chunk
- Doesn’t synthesize across chunks

Step 4: Generation++

- Better prompt structure
- Add citations requirement
- Use system instructions for grounding
- Post-processing (verification, formatting)

Retrieval vs. Generation

- Retrieval = getting the right information
- Generation = using it correctly

- Retrieval failure → Generation failure

Let's Code

? Quiz Time! ?

Test Your Knowledge & Have Fun!

Get Ready to Participate!

Q1: You're chunking a 200-page legal contract. Sentence-level chunks (1–2 sentences each) give poor retrieval results. What's the most likely reason?

A) The embedding model doesn't support legal text

B) Individual sentences lack enough context to produce meaningful embeddings

C) The vector database can't handle that many chunks

D) Sentence chunking always outperforms paragraph chunking

Q1: You're chunking a 200-page legal contract. Sentence-level chunks (1–2 sentences each) give poor retrieval results. What's the most likely reason?

B) Individual sentences lack enough context to produce meaningful embeddings

Very small chunks lose surrounding context. A sentence like "This shall not apply in cases described above" is meaningless without what came before. Semantic clarity requires enough context within each chunk.

Q2: Your RAG pipeline returns the right documents (high recall) but the answer still misses key information. What should you check first?

A) Increase the embedding model dimensions

B) The chunk boundaries — relevant information may be split across two chunks

C) Switch to a larger LLM

D) Add more documents to the index

Q2: Your RAG pipeline returns the right documents (high recall) but the answer still misses key information. What should you check first?

B) The chunk boundaries — relevant information may be split across two chunks

High recall means the right documents were retrieved, but if a critical fact straddles a chunk boundary, neither chunk contains the complete information. Overlapping chunks or logical chunking at natural boundaries fixes this.

RAG Alternatives & OOTB

Long context, fine-tuning, frameworks

Alternative 1: Long Context Windows

The pitch: modern models (Gemini 2.5 Pro, Claude, etc.) offer 1M+ token context windows. Why not just stuff everything in?

Pros

→ Zero indexing pipeline

→ Simpler architecture

→ No chunking decisions

Cons

→ Expensive per call

→ Latency grows with length

→ Still capped at 1–2M tokens

→ Hard to cite sources precisely

"Lost in the Middle" effect

Accuracy drops when the answer is buried in the center of a long prompt. Models are biased toward beginning and end. Even million-token windows don't guarantee uniform comprehension.

Alternative 2: Domain Fine-Tuning

	Fine-Tuning	RAG
When to use	Style, tone, format, vocabulary	Private data, facts, dynamic content
Data freshness	Frozen at training time	Updated at retrieval time
Cost	High upfront (GPU training)	Pay per retrieval call
Hallucination risk	Can memorize & confuse PII	Grounded — lower risk
Data changes?	Requires full retraining	Just update the index

Best of both worlds: fine-tune the model on style, use RAG for knowledge.

RAG Frameworks - pick by tradeoff

LangChain

✓ Huge ecosystem, composable graph model, great for agents & chat memory

✗ Steep learning curve, verbose, performance overhead

Best for: Complex multi-step pipelines

LlamaIndex

✓ Best-in-class ingestion pipeline, rich index types, lightweight

✗ Rapid API changes, smaller community than LangChain

Best for: Document-centric RAG

OpenAI Retrieval

✓ Zero-ops: auto-chunking, embedding, vector store. Great defaults.

✗ Cloud lock-in, limited customization (no re-rankers)

Best for: Quick prototype, GPT-native

Vertex AI Search

✓ Enterprise-grade, GCP integration, multi-TB corpora, compliance

✗ GCP lock-in, opaque internals

Best for: Large enterprise on GCP

Yuval-uating RAG Systems

Because if you can't measure it, you can't improve it

Why RAG evaluation is hard

A RAG system has two moving parts — and either can be the source of failure.

Retrieval fails

The right documents were never retrieved. The LLM had no chance — it was working with bad inputs.

Generation fails

The right docs were retrieved, but the LLM ignored them, hallucinated, or answered incorrectly.

Both fail

Compound failure. Irrelevant docs + a confused model. These are the hardest to debug.

The four metrics you need

Context Recall — Retrieval
Did we retrieve all the relevant docs?

Context Precision — Retrieval
Are the retrieved docs actually relevant?

Answer Faithfulness — Generation
Is the answer supported by the retrieved context?

Answer Relevance — Generation
Does the answer actually address the question?

The RAG Triad (Your Diagnostic Framework)

- Three metrics that map to two pipeline stages

Metric	Measures	When it fails, tune...
Context Relevance	Are retrieved docs actually about the query?	
Faithfulness	Is the answer grounded in retrieved context?	
Answer Relevance	Does the answer address the question asked?	

The RAG Triad (Your Diagnostic Framework)

- Three metrics that map to two pipeline stages

Metric	Measures	When it fails, tune...
Context Relevance	Are retrieved docs actually about the query?	Embedding model, chunking strategy, top-k
Faithfulness	Is the answer grounded in retrieved context?	LLM selection, temperature, grounding prompt
Answer Relevance	Does the answer address the question asked?	Prompt template, in-context examples

Evaluating Retrieval in Isolation

- Context Recall (existential): Did we get all the relevant docs?
  - Missing relevant docs = generation fails regardless of LLM quality
  - Formula: relevant retrieved ÷ total relevant
- Context Precision (signal quality): Are the retrieved docs actually relevant?
  - Noise increases hallucination risk
  - Formula: relevant retrieved ÷ total retrieved

Evaluating Retrieval in Isolation

- Context Recall (e
  - Mis
-
  ac
  - Nois
  - Formula: relevant retri

the relevant docs?

LM

Retrieval is often the real bottleneck. Legal RAG Bench: switching the embedding model improved correctness by 17.5 points - before touching the LLM.

Where Do the Golden Labels Come From?

Three approaches, ranked by cost:

1. Synthetic (cheapest): LLM-generated Q&A pairs

- Feed each chunk to an LLM: "Generate 3 questions that this passage answers."
- Result: question → chunk mapping = instant ground truth.
- Caveat: the LLM only generates questions for information it can see - it won't generate the hard questions your users actually ask.

Where Do the Golden Labels Come From?

Three approaches, ranked by cost:

1. Synthetic (cheapest): LLM-generated Q&A pairs
2. Semi-synthetic: mine your logs

- If you have user queries from a previous system (search logs, support tickets, chat history), pair them with the chunks that should have answered them.
- Best of both worlds: real questions, manageable labeling effort.

Where Do the Golden Labels Come From?

Three approaches, ranked by cost:

1. Synthetic (cheapest): LLM-generated Q&A pairs
2. Semi-synthetic: mine your logs
3. Human annotation (most expensive, most accurate)

- Domain experts write questions against documents.
- Required for high-stakes domains (legal, medical). Not a first step.

Evaluating Generation in Isolation

- Faithfulness (grounding check): Is every claim in the answer supported by the retrieved context?
  - Low faithfulness = LLM is hallucinating or ignoring docs
  - Fix: stronger grounding instructions, lower temperature, different model
- Answer Relevance (usefulness check): Does the answer address what the user actually asked?
  - Low relevance = off-topic or evasive answers
  - Fix: better prompt template, in-context examples

End-to-End vs Component Evaluation

Should you evaluate end-to-end or component by component?

	End-to-End	Component-Level
What you measure	Final answer quality	Retriever or generator in isolation
When to use	Regression tests, benchmarks	Debugging a specific failure
Limitation	Hides which component failed	Requires test sets per component

End-to-End vs Component Evaluation

Should you evaluate by component?

W m		
W		
Limitation	compo	or

End-to-end pass rates hide root causes. If your score drops, component eval tells you whether retrieval degraded or the LLM changed behavior

RAG Benchmarks - Where Systems Get Tested

General-purpose

- **RAGBench** 100K examples, 5 domains, multi-hop questions
- **LongBench v2**: 10K–50K token contexts, "lost in the middle" detection
- **MRCRv2**: multi-hop reading comprehension; where cheap models fail hardest

Domain-specific

- **Legal RAG Bench**: 4,876 passages, expert questions, retrieval is the ceiling
- **MIRAGE**: 7,663 medical QA questions, 5 datasets
- **FinanceBench**: 10-K filings; vector RAG ~50%, vectorless ~99%

Robustness

- **RGB**: noise robustness, negative rejection, counterfactual resistance
- **URAG**: retrieval noise vs model uncertainty/confidence

Problems with RAG Benchmarks

Local questions, local answers

- Assuming an answer lies in a certain chunk

Problems with RAG Benchmarks

Local questions, local answers

- Assuming an answer lies in a certain chunk

Multi-hop questions are not realistic

- *"If my future wife has the same first name as the 15th first lady of the United States' mother and her surname is the same as the second assassinated president's mother's maiden name, what is my future wife's name?" (from: FRAMES)*

Problems with RAG Benchmarks

Local questions, local answers

- Assuming an answer lies in a certain chunk

Multi-hop questions are not realistic

- *"If my future wife has the same first name as the 15th first lady of the United States' mother and her surname is the same as the second assassinated president's mother's maiden name, what is my future wife's name?" (from: FRAMES)*

No holistic way to test the entire system

- Retrieval-only benchmarks
- Generation-only benchmark (grounding)
- What about chunking? What about parsing?

Problems with RAG Benchmarks

Local questions, local answers

- Assuming an answer lies in a certain chunk

Multi-hop questions are not realistic

- *"If my future wife has the same first name as the 15th first lady of the United States' mother and her surname is the same as the second assassinated president's mother's maiden name, what is my future wife's name?" (from: FRAMES)*

No holistic way to test the entire system

- Retrieval-only benchmarks
- Generation-only benchmark (grounding)
- What about chunking? What about parsing?

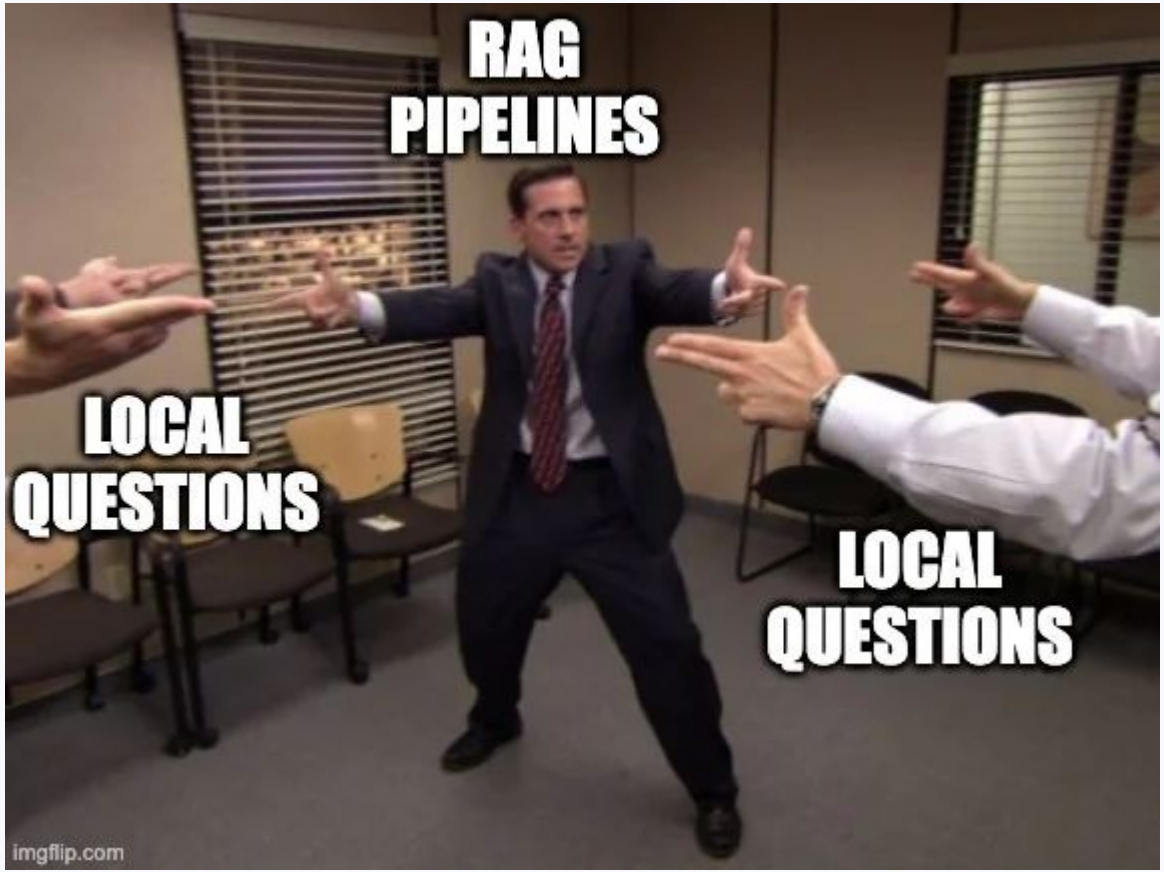
Benchmarks do not correlate with real world data!

- Real world data is messier
- Each data is different, and not generalizes well

The Vicious Cycle

1. Build RAG systems for flawed benchmarks
2. Celebrate our awesome benchmark scores
3. Watch real users struggle
4. Create new benchmarks with the same problems
5. Rinse and repeat

The Vicious Cycle



Learn More

EPISODE 6 • 43 min

RAG Is Not Solved – Your Evaluation Just Sucks

Yuval and Niv from AI21 discuss why most RAG systems fail in the real world, despite passing benchmarks. They cover issues like flawed benchmarks, poor data...

July 29, 2025

? Quiz Time! ?

Test Your Knowledge & Have Fun!

Get Ready to Participate!

Q3: You've indexed 10,000 documents. For a given query, context recall is 0.4. What is the most accurate interpretation?

A) 40% of retrieved chunks were relevant to the query

B) Only 40% of the actually-relevant documents were retrieved

C) The LLM used 40% of the retrieved context in its answer

D) The answer was correct 40% of the time

Q3: You've indexed 10,000 documents. For a given query, context recall is 0.4. What is the most accurate interpretation?

B) Only 40% of the actually-relevant documents were retrieved

Context recall measures whether the retriever found all the relevant documents - 60% of needed information was never retrieved, so the generator had no chance of being complete.

Q4: Your RAG system has perfect context recall (1.0) but faithfulness is 0.5. Where should you investigate first?

A) The chunking strategy — chunks are too large

B) The embedding model — it's not domain-adapted

C) The generator: LLM prompt, temperature, or model selection

D) The top-k parameter — retrieving too many chunks

Q4: Your RAG system has perfect context recall (1.0) but faithfulness is 0.5. Where should you investigate first?

C) The generator: LLM prompt, temperature, or model selection

Perfect recall means all relevant docs were retrieved — retrieval is not the problem. Faithfulness = is the answer grounded in context? 0.5 means the generator is hallucinating or ignoring context

Summary

What we did today?

RAG: pre-process → chunk → embed → retrieve → generate. Each step has failure modes and levers.

Evaluating RAG: The RAG Triad: context relevance, faithfulness, answer relevance. Evaluate components in isolation to pinpoint failures. Retrieval is usually the bottleneck.

The best RAG system is the one you can measure, debug, and improve. Build for observability from day one